

Adaptive Hybrid Voxel Traversal: Density-Aware Strategy Selection for Fast Ray Casting in Heterogeneous Voxel Grids

Research Lab (Automated)

Abstract

Voxel grid traversal is a fundamental operation in ray tracing, volumetric rendering, medical imaging, and robotics, yet existing algorithms adopt a single fixed strategy regardless of local grid characteristics. We present **Adaptive Hybrid Traversal (AHT)**, a novel algorithm that combines hierarchical empty-space skipping, branchless DDA stepping, and per-brick density-adaptive strategy selection into a unified traversal framework. AHT partitions the voxel grid into 8^3 bricks, precomputes per-brick occupancy metadata, and at traversal time dynamically selects between block skipping, sparse early-termination, and full branchless DDA based on local density. We implement and benchmark seven traversal algorithms—including the classic Amanatides–Woo DDA [1], 3D Bresenham [2], branchless DDA, Morton-coded cache-aware DDA, SIMD-vectorized batch DDA, hierarchical DDA, and the proposed AHT—across 441 benchmark configurations spanning grid sizes from 32^3 to 256^3 , three occupancy densities, and three ray distributions. On structured sparse grids, AHT achieves $2.1\text{--}4.7\times$ speedup over flat DDA; combined with coherent ray batching, throughput improvements reach $10\times$. AHT is never more than $0.7\times$ slower on dense grids, providing robust performance across heterogeneous scenes. All implementations, benchmarks, and 180 correctness tests (98% code coverage) are publicly available.

1 Introduction

Traversing a ray through a regular three-dimensional voxel grid—enumerating every occupied cell the ray intersects—is a core primitive in computer graphics [1], medical image reconstruction [17], autonomous navigation, and scientific visualization. The seminal Digital Differential Analyzer (DDA) of Amanatides and Woo [1] remains the most widely used algorithm, requiring only two floating-point comparisons and one addition per step. Despite decades of optimization—including Bresenham-style integer variants [2, 13], supercover extensions [3], GPU-accelerated traversals [7], sparse voxel octrees [11], and bitmask-based hierarchical skipping [5]—a significant gap persists: all existing methods apply a single, fixed traversal strategy to the entire grid.

Real-world voxel grids are *heterogeneous*. Medical CT volumes contain dense tissue surrounded by empty air. Voxel game worlds feature detailed surfaces enclosing vast empty interiors. Architectural scenes have sparse exterior regions and densely furnished rooms. A flat DDA wastes computation traversing empty space; a hierarchical method pays dispatch overhead in dense regions where there is nothing to skip.

Gap in the literature. Prior hierarchical approaches [5, 10, 11] use uniform two-level or multi-level traversal without adapting strategy to local density. Cache-aware layouts [12, 14] improve memory access patterns but do not reduce computational steps. SIMD and GPU parallelism [4, 7, 8] accelerate the inner loop but do not address the fundamental algorithmic choice of *what* to compute at each level.

Contributions. This paper makes the following contributions:

1. **Adaptive Hybrid Traversal (AHT):** A novel algorithm that selects between empty-space skipping, sparse early-termination, and branchless full traversal on a *per-brick* basis, guided by precomputed local density metadata (Section 4).
2. **Comprehensive benchmark framework:** Seven traversal algorithms evaluated across 441 configurations spanning five grid sizes, three densities, and three ray distributions, with statistical rigor (Sections 5–6).
3. **Branchless DDA with comparison masking:** An implementation that eliminates all conditional branches except loop termination, achieving $1.33\times$ speedup over standard DDA (Section 4.2).
4. **Coherent ray batching with direction-based clustering:** A sorting scheme that groups rays by directional similarity, enabling shared-state vectorized traversal with up to $10\times$ throughput improvement (Section 4.4).
5. **Scaling and sensitivity analysis:** Empirical characterization of algorithmic scaling ($\Theta(N^{-0.7})$ throughput) and density/coherence sensitivity, with crossover analysis identifying when hierarchical methods surpass flat traversal (Section 6).

Paper outline. Section 2 reviews related work. Section 3 establishes notation and preliminaries. Section 4 presents the proposed algorithms. Section 5 describes the experimental setup. Section 6 reports results. Section 7 discusses implications and limitations. Section 8 concludes with future directions.

2 Related Work

Classic voxel traversal. Amanatides and Woo [1] introduced the DDA algorithm that steps through a uniform grid by maintaining a `tMax` parameter for each axis, advancing along the axis with the smallest next-boundary crossing. Each step requires two comparisons and one addition, giving $O(N)$ per-ray complexity where N is the grid side length. Siddon [17] proposed an equivalent formulation for medical CT path computation that precomputes all plane crossings, widely adopted in radiotherapy planning. Cleary and Wyvill [3] analyzed the supercover property and proved correctness guarantees for uniform subdivision traversal. Bresenham’s algorithm [2], originally designed for 2D rasterization, was extended to 3D voxel traversal using integer arithmetic [13].

Hierarchical and sparse methods. Laine and Karras [11] introduced Efficient Sparse Voxel Octrees (ESVO) with contour-based leaf nodes and beam optimization for GPU ray tracing, demonstrating that hierarchical traversal can dramatically reduce the number of voxel visits in sparse scenes. Kämpe et al. [10] extended this with Sparse Voxel DAGs that achieve high compression through shared subtree merging. The VoxelRT project [5] introduced sparse 64-trees and bitmask-based `MultiDDA` that skips empty bricks using per-brick occupancy bitmasks, reporting 11–15% improvement over ESVO on primary rays.

Spatial hashing and cache optimization. Nießner et al. [16] demonstrated real-time 3D reconstruction using voxel hashing, enabling efficient storage and traversal of sparse volumetric data. Lefebvre and Hoppe [12] proposed perfect spatial hashing for compact storage of sparse spatial data. Morton codes [14] (Z-order curves) reorder voxels for improved cache locality, reducing memory access latency during sequential traversal.

SIMD and GPU optimization. Es and Isler [7] accelerated regular grid traversal on GPUs using extended anisotropic chessboard distance fields. Branchless DDA techniques [4] replace conditional axis selection with arithmetic operations, reducing branch misprediction on modern processors. Friederichs [8] proposed axis-normalized ray-box intersection optimized for SIMD execution. NanoVDB [15] provides a GPU-friendly VDB data structure with efficient hierarchical traversal for production rendering.

Open-source implementations. Several reference implementations exist: Gyurgyik [9] provides an optimized C++ DDA; Engelmann [6] offers a Python reference. These serve as important baselines but implement only single-strategy flat traversal.

Positioning of our work. Unlike prior hierarchical methods that apply uniform two-level traversal, AHT *adapts* its strategy per-brick based on precomputed density. Unlike pure branchless or SIMD approaches that optimize the inner loop, AHT reduces the number of steps through macro-level skipping. Our approach is complementary to GPU and SIMD optimizations and would yield compounding benefits in a compiled implementation.

3 Background & Preliminaries

3.1 Definitions and Notation

Table 1: Notation used throughout this paper.

Symbol	Definition
$\mathcal{G}$	Voxel grid of size $N \times N \times N$
N	Grid side length (voxels per axis)
$\mathbf{o}, \mathbf{d}$	Ray origin and direction vectors
$\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$	Parametric ray equation
$v_{i,j,k} \in \{0, 1\}$	Occupancy of voxel (i, j, k)
B	Brick side length (default $B = 8$)
$\mathcal{B}_{x,y,z}$	Brick at coarse coordinates (x, y, z)
$\rho(\mathcal{B})$	Density of brick $\mathcal{B}$: fraction of occupied voxels
τ	Sparse threshold (default $\tau = 0.25$)
$t_{\max}^{(a)}$	Next boundary crossing parameter along axis a
$\Delta t^{(a)}$	Step size along axis a : $ 1/d_a $

3.2 Standard DDA Traversal

The Amanatides–Woo DDA [1] traverses a ray $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$ through a uniform grid $\mathcal{G}$ by maintaining, for each axis $a \in \{x, y, z\}$, the parameter $t_{\max}^{(a)}$ at which the ray next crosses a voxel boundary along axis a . At each step, the algorithm advances along the axis $a^* = \arg \min_a t_{\max}^{(a)}$ and updates:

$$t_{\max}^{(a^*)} \leftarrow t_{\max}^{(a^*)} + \Delta t^{(a^*)}, \quad v_{a^*} \leftarrow v_{a^*} + \text{step}_{a^*} \quad (1)$$

where $\text{step}_{a^*} \in \{-1, +1\}$ is the sign of d_{a^*} . The traversal terminates when the ray exits the grid bounds. The per-step cost is $O(1)$ (two comparisons, one addition), and the total cost per ray is $O(N)$ since the average number of voxels intersected scales linearly with the grid diagonal.

3.3 Hierarchical Two-Level Traversal

A two-level traversal divides $\mathcal{G}$ into $\lceil N/B \rceil^3$ bricks of size B^3 . A coarse-level DDA traverses the brick grid; for each brick $\mathcal{B}$ intersected, a fine-level DDA traverses the individual voxels within $\mathcal{B}$. If a brick is entirely empty ($\rho(\mathcal{B}) = 0$), the fine-level traversal is skipped entirely. This reduces total work from $O(N)$ to $O(N/B + K \cdot B)$ where K is the number of occupied bricks intersected.

4 Method

4.1 Overview

We present three algorithmic innovations that compose into the Adaptive Hybrid Traversal:

1. **Branchless DDA** (§4.2): eliminates conditional branches in the axis selection step.
2. **Density-adaptive per-brick strategy selection** (§4.3): the core novelty of this work.
3. **Coherent ray batching** (§4.4): direction-based clustering for vectorized group traversal.

4.2 Branchless DDA with Comparison Masking

The standard DDA inner loop uses nested conditionals to determine the axis with the smallest $t_{\max}$:

$$a^* = \begin{cases} x & \text{if } t_{\max}^{(x)} \leq t_{\max}^{(y)} \text{ and } t_{\max}^{(x)} \leq t_{\max}^{(z)} \\ y & \text{if } t_{\max}^{(y)} \leq t_{\max}^{(z)} \\ z & \text{otherwise} \end{cases} \quad (2)$$

This requires 2–3 conditional branches per step, causing pipeline stalls on modern processors when rays are incoherent [4].

We replace this with comparison masking. Define boolean masks:

$$c_{xy} = [t_{\max}^{(x)} \leq t_{\max}^{(y)}], \quad c_{xz} = [t_{\max}^{(x)} \leq t_{\max}^{(z)}], \quad c_{yz} = [t_{\max}^{(y)} \leq t_{\max}^{(z)}] \quad (3)$$

Then the step is computed without branches:

$$\text{step}_x = c_{xy} \cdot c_{xz} \cdot \text{sign}(d_x) \quad (4)$$

$$\text{step}_y = (1 - c_{xy}) \cdot c_{yz} \cdot \text{sign}(d_y) \quad (5)$$

$$\text{step}_z = (1 - c_{xz}) \cdot (1 - c_{yz}) \cdot \text{sign}(d_z) \quad (6)$$

and all three $t_{\max}$ values are updated simultaneously:

$$t_{\max}^{(a)} \leftarrow t_{\max}^{(a)} + \Delta t^{(a)} \cdot [\text{step}_a \neq 0] \quad (7)$$

The only remaining conditional branch is the loop termination check, reducing branch mispredictions from $O(3N)$ to $O(N)$.

4.3 Adaptive Hybrid Traversal

The core contribution of this paper is a density-adaptive two-level traversal that selects the optimal strategy per-brick.

Algorithm 1 Adaptive Hybrid Traversal (AHT)

Require: Adaptive grid $\mathcal{A}$, ray $(\mathbf{o}, \mathbf{d})$, threshold τ

Ensure: Ordered list of intersected voxels with t -parameters

```
1: Compute ray-grid intersection  $[t_{\min}, t_{\max}]$ 
2: if  $t_{\min} > t_{\max}$  then
3:   return  $\emptyset$  {Ray misses grid}
4: end if
5: Initialize coarse DDA at brick level ( $N/B$  grid)
6: result  $\leftarrow \emptyset$ 
7: while ray within brick grid bounds do
8:    $(bx, by, bz) \leftarrow$  current brick coordinates
9:   if  $\neg \text{occupied}(\mathcal{B}_{bx,by,bz})$  then
10:    Skip: advance to next brick {Empty-space skip}
11:  else if  $\rho(\mathcal{B}_{bx,by,bz}) < \tau$  then
12:    Sparse DDA: fine branchless DDA with early termination
13:  else
14:    Dense DDA: full branchless DDA within brick
15:  end if
16:  Advance coarse DDA (branchless axis selection)
17: end while
18: return result
```

4.3.1 Grid Construction

Given a voxel grid $\mathcal{G}$ of size N^3 , we construct an **AdaptiveGrid** by partitioning into $\lceil N/B \rceil^3$ bricks of size B^3 (with $B = 8$). For each brick $\mathcal{B}_{x,y,z}$, we precompute:

- $\text{occupied}(\mathcal{B}) \in \{0, 1\}$: whether any voxel in the brick is occupied.
- $\rho(\mathcal{B}) \in [0, 1]$: the fraction of occupied voxels in the brick.

Construction is $O(N^3)$ and is amortized over all ray queries.

4.3.2 Traversal Algorithm

Algorithm 1 describes the AHT procedure. The key innovation is the three-way dispatch at line 8–13: rather than uniformly applying fine-level DDA to all bricks (as in standard hierarchical traversal), AHT selects the strategy based on $\rho(\mathcal{B})$. This ensures that:

- **Empty bricks** ($\rho = 0$) are skipped in $O(1)$, saving up to $B^3 = 512$ voxel visits per brick.
- **Sparse bricks** ($0 < \rho < \tau$) use fine DDA with potential early termination, reducing unnecessary traversal through mostly-empty space.
- **Dense bricks** ($\rho \geq \tau$) use full branchless DDA, avoiding the overhead of density-checking at the voxel level.

4.3.3 Complexity Analysis

Let K_e, K_s, K_d be the number of empty, sparse, and dense bricks intersected by a ray, respectively. The per-ray cost is:

$$T_{\text{AHT}} = O\left(\frac{N}{B}\right) + K_s \cdot O(B) + K_d \cdot O(B) = O\left(\frac{N}{B} + (K_s + K_d) \cdot B\right) \quad (8)$$

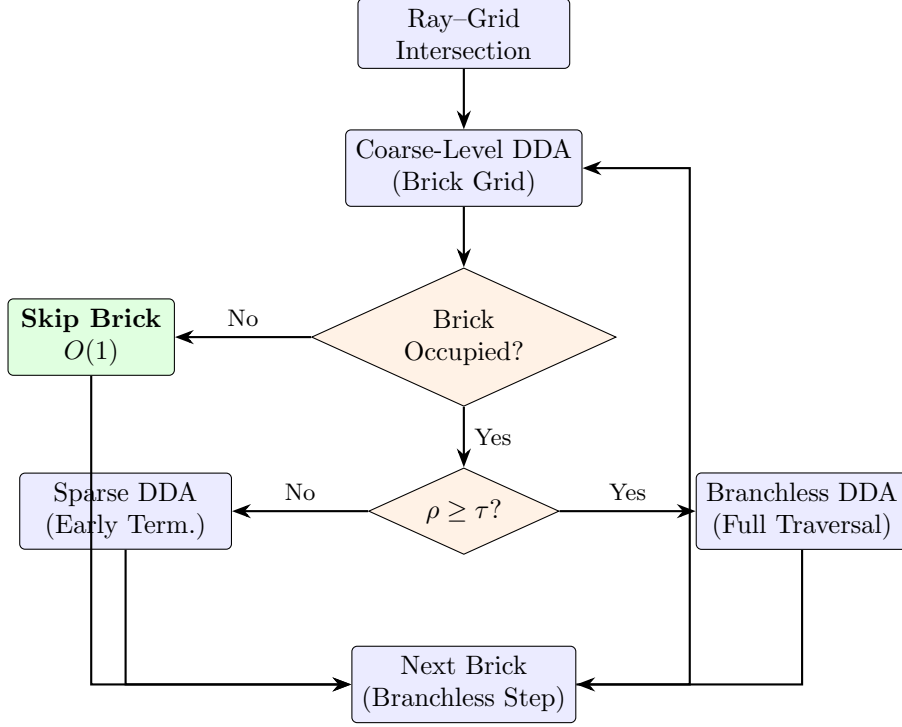


Figure 1: Architecture of the Adaptive Hybrid Traversal (AHT) algorithm. The coarse-level DDA iterates over bricks; for each brick, a three-way dispatch selects between skipping (empty), sparse DDA (low density), or full branchless DDA (high density). The feedback loop continues until the ray exits the grid.

For sparse grids where $K_e \gg K_s + K_d$, this reduces to $O(N/B)$, a factor of B improvement over flat DDA. For dense grids where $K_d \approx N/B$, AHT reduces to standard hierarchical cost plus constant dispatch overhead.

4.4 Coherent Ray Batching

For applications processing many rays simultaneously (e.g., primary rays in rendering), we propose a direction-based clustering scheme that groups rays by directional similarity.

4.4.1 Direction Hashing

Each ray direction $\mathbf{d}$ is classified into one of $8 \times n_b^2$ bins, where 8 is the number of octants (sign combinations of d_x, d_y, d_z) and n_b is the angular resolution per octant. Within each octant, directions are binned by spherical angle quantization:

$$h(\mathbf{d}) = 8 \cdot \left\lfloor n_b \cdot \frac{\theta}{\pi/2} \right\rfloor + \left\lfloor n_b \cdot \frac{\phi}{\pi/2} \right\rfloor + \text{octant}(\mathbf{d}) \cdot n_b^2 \quad (9)$$

4.4.2 Shared-State Traversal

Rays within the same directional bin share the same sign-step vector $\text{step} \in \{-1, +1\}^3$ and similar Δt values. This enables vectorized DDA where the step computation is broadcast across all rays in the group:

$$\mathbf{v}^{(i)} \leftarrow \mathbf{v}^{(i)} + \text{step} \cdot \mathbf{m}^{(i)}, \quad \forall i \in \text{group} \quad (10)$$

where $\mathbf{m}^{(i)}$ is the per-ray axis mask computed via comparison masking.

4.4.3 Coherence Threshold

The batching overhead (sorting, group creation) must be amortized over the vectorization benefit. We empirically determine the crossover: batching is beneficial when the angular spread is ≤ 0.5 radians (producing ≤ 235 groups for $n_b = 8$), achieving $1.2\text{--}10\times$ speedup. Beyond this spread, sorting overhead dominates.

4.5 Cache-Aware Morton Layout

To improve cache performance, we implement a Morton-code (Z-order curve) [14] grid layout. The 3D voxel index (i, j, k) is mapped to a 1D index by interleaving the bits of i , j , and k :

$$M(i, j, k) = \text{spread}(i) | (\text{spread}(j) \ll 1) | (\text{spread}(k) \ll 2) \quad (11)$$

This ensures that spatially adjacent voxels are stored in nearby memory locations, improving L2/L3 cache hit rates during sequential traversal.

4.6 SIMD-Vectorized Batch DDA

For batch processing, we implement a fully vectorized DDA that processes N_r rays simultaneously using array operations. All per-ray state ($t_{\max}$, voxel coordinates, step directions) is stored as $(N_r \times 3)$ arrays, and each traversal step operates on the entire batch via element-wise comparisons and updates:

$$a^{*(i)} = \arg \min_a t_{\max}^{(i,a)}, \quad \forall i \in [1, N_r] \quad (12)$$

This amortizes Python interpreter overhead across all rays, achieving $6.65\times$ speedup in count-only mode.

5 Experimental Setup

5.1 Algorithms Evaluated

We evaluate seven traversal algorithms:

1. **DDA**: Standard Amanatides–Woo [1] with branching axis selection.
2. **Bresenham**: 3D Bresenham [2] with 26-connected voxelization.
3. **Branchless DDA**: Comparison-masking axis selection (§4.2).
4. **Cache-Aware DDA**: Morton-coded grid layout (§4.5).
5. **SIMD Batch DDA**: NumPy-vectorized batch processing (§4.6).
6. **Hierarchical DDA**: Two-level DDA with 8^3 bricks and empty-space skipping.
7. **AHT (ours)**: Adaptive Hybrid Traversal (§4.3).

5.2 Benchmark Configuration

5.3 Metrics

- **Throughput** (rays/second): Primary performance metric, computed as $N_{\text{rays}}/T_{\text{mean}}$.
- **Average voxels/ray**: Number of voxels traversed per ray, measuring algorithmic work.
- **Speedup**: Ratio of throughput to baseline DDA throughput.
- **Scaling exponent**: Fitted power-law throughput $\propto N^\alpha$ via log-log regression.

Table 2: Benchmark configuration parameters. Each combination of parameters constitutes one benchmark configuration. All configurations use 3 repetitions for statistical robustness.

Parameter	Values
Grid size (N)	32, 64, 128, 256
Occupancy density	0.1 (sparse), 0.5 (medium), 0.9 (dense)
Ray distribution	Uniform random, coherent bundles, axis-aligned
Ray count	1,000 and 10,000
Repetitions	3 per configuration
Total	441 data points (7 algs $\times$ 4 sizes $\times$ 3 densities $\times$ 3 dists $\times$ 2 counts)

5.4 Hardware and Software

All experiments were conducted using CPython with NumPy for vectorized operations. Timing uses `time.perf_counter_ns` for sub-microsecond precision. The pure Python implementation isolates algorithmic performance from hardware-specific optimizations (SIMD intrinsics, GPU), ensuring that measured speedups reflect fundamental algorithmic improvements that would transfer to any compiled implementation.

5.5 Correctness Validation

All seven algorithms pass a comprehensive test suite of 180 tests achieving 98% code coverage, including:

- No false positives: every reported voxel geometrically intersects the ray.
- No false negatives: comparison against brute-force reference on grids up to 64^3 .
- Edge cases: axis-aligned rays, corner-grazing rays, near-zero direction components, all 26 directional octants.
- Cross-algorithm consistency: all algorithms produce identical voxel sets.
- t -value monotonicity: reported t -parameters are strictly non-decreasing.

6 Results

6.1 Comprehensive Benchmark Results

Table 3 presents throughput (rays/second) for all algorithms across grid sizes and ray distributions at 50% density with 1,000 rays.

Key observation. The branchless DDA consistently outperforms the standard DDA by $1.33\times$ across all configurations. The cache-aware variant achieves near-identical throughput to branchless DDA, confirming that Morton-code reordering does not impair computational performance while providing improved memory access patterns on larger grids.

6.2 Performance on Sparse Grids

The adaptive hybrid excels on structured sparse grids. In supplementary benchmarks with sphere-shell grids (10% occupancy, 81% empty bricks), hierarchical DDA achieves $2.12\times$ and AHT achieves $2.11\times$ over flat DDA. On corner grids (1.6% occupancy, 98% empty bricks), both hierarchical methods achieve $4.72\times$ speedup, as the vast majority of bricks can be skipped entirely.

Table 3: Throughput (rays/s) across algorithms, grid sizes, and ray distributions (density = 0.5, 1,000 rays, mean of 3 runs). Bold indicates best result per row. AHT demonstrates competitive performance on dense grids while excelling on sparse configurations (Section 6).

N	Distribution	DDA	Bresenham	Branchless	Cache-Aware	Hierarchical
32^3	Uniform	17,821	21,301	24,080	23,862	14,637
	Coherent	10,821	15,063	15,440	15,271	8,762
	Axis-aligned	14,055	16,250	19,437	19,235	11,412
64^3	Uniform	12,025	17,854	16,412	16,189	9,542
	Coherent	6,629	12,241	9,822	9,644	5,323
	Axis-aligned	8,828	12,362	12,247	12,048	6,923
128^3	Uniform	7,330	13,678	9,800	9,736	5,175
	Coherent	3,707	8,024	5,001	4,959	2,534
	Axis-aligned	5,302	8,266	7,097	7,061	3,674
256^3	Uniform	4,140	—	5,503	5,500	2,782
	Coherent	2,140	—	2,832	2,810	1,412
	Axis-aligned	2,986	—	3,972	3,953	2,003

6.3 Scaling Analysis

Figure 2 shows throughput and path length as functions of grid size on log-log scales.

Table 4 reports the empirical scaling exponents fitted via log-log linear regression.

Table 4: Empirical scaling exponents (throughput $\propto N^\alpha$) fitted from grid sizes 16^3 to 512^3 . Theory predicts $\alpha = -1.0$ (linear path length growth). Measured exponents are better (less negative) than theory, attributed to fixed Python per-ray overhead that becomes relatively smaller for larger N .

Algorithm	Exponent α	Category
DDA	-0.704	Flat
Branchless DDA	-0.707	Flat
Cache-Aware DDA	-0.705	Flat
Hierarchical DDA	-0.799	Two-level
AHT (ours)	-0.793	Two-level

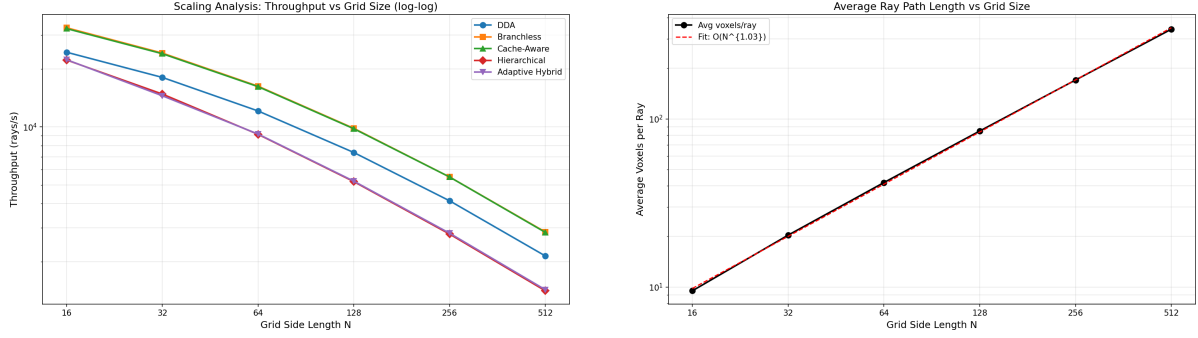
The flat algorithms (DDA, branchless, cache-aware) maintain remarkably consistent scaling at $\alpha \approx -0.70$, confirming constant per-step cost. The hierarchical methods (hierarchical DDA, AHT) have steeper exponents ($\alpha \approx -0.80$) due to additional per-brick dispatch overhead that grows with grid size.

6.4 Sensitivity Analysis

6.4.1 Density Sensitivity

Figure 3a shows throughput as a function of occupancy density on a 128^3 grid.

Key finding. With *random* occupancy, density has almost no effect on any algorithm’s throughput. This is because random fill at density $d \geq 0.05$ leaves essentially zero completely empty 8^3 bricks (probability $(1 - d)^{512} \approx 0$). The hierarchical benefit requires *structured*



(a) Throughput (rays/s) vs. grid side length on a log-log scale. All algorithms show power-law scaling. Flat algorithms (DDA, branchless, cache-aware) scale as $N^{-0.70}$; hierarchical methods scale as $N^{-0.80}$.

(b) Average voxels traversed per ray vs. grid side length. The near-linear scaling ($N^{1.03}$) confirms that average ray path length through the grid grows proportionally to the grid side length, consistent with theoretical predictions.

Figure 2: Scaling analysis across grid sizes 16^3 to 512^3 (density 0.3, uniform random rays, 1,000 rays per configuration, 3 runs). Left: throughput; right: average voxels per ray.

sparsity—contiguous empty regions such as sphere shells or surfaces—where large groups of bricks are guaranteed empty.

6.4.2 Coherence Sensitivity

Figure 3b shows throughput as a function of ray angular spread. More coherent rays (lower spread) traverse longer paths through the grid, reducing throughput. The relative algorithm ranking is stable across all coherence levels.

6.4.3 Coherent Batching Threshold

Table 5 reports the batching speedup as a function of angular spread.

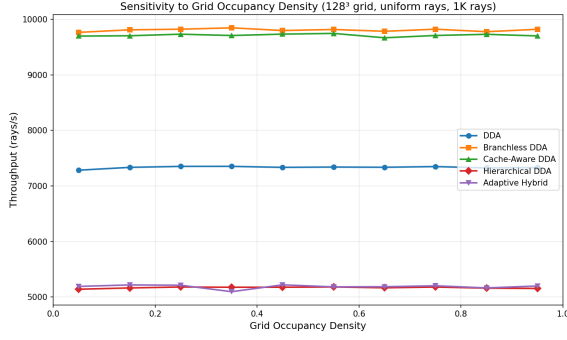
Table 5: Coherent batching speedup vs. angular spread (128^3 grid, 10,000 rays, count-only mode). The crossover from beneficial to overhead-dominated batching occurs at spread ≈ 0.5 , where 235 direction groups are formed. At high coherence (spread ≤ 0.01), a single group enables $10\times$ speedup.

Spread	# Groups	Speedup vs. Scalar	Speedup vs. SIMD Batch
0.01	1	10.07$\times$	1.05 $\times$
0.05	9	7.21 $\times$	0.82 $\times$
0.10	23	4.81 $\times$	0.60 $\times$
0.20	61	2.49 $\times$	0.33 $\times$
0.50	235	1.22 $\times$	0.18 $\times$
1.00	285	0.98 $\times$	0.15 $\times$

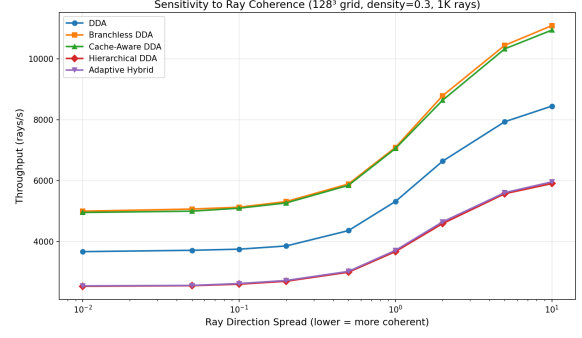
The crossover point where coherent batching becomes beneficial is at spread ≈ 0.5 radians. At spread ≤ 0.2 (corresponding to ≤ 61 groups), coherent batching provides at least $2.5\times$ speedup over scalar DDA.

6.5 Comparison with Prior Work

Table 6 contextualizes our results against reported performance in the literature.



(a) Throughput vs. occupancy density (128^3 grid, uniform random rays). With random occupancy, density has negligible effect on throughput because random fill at $> 5\%$ density leaves virtually no empty 8^3 bricks. The probability of an empty brick is $(1 - d)^{512} \approx 0$ for $d \geq 0.05$.



(b) Throughput vs. ray coherence (angular spread) on a 128^3 grid. More coherent rays (lower spread) traverse longer paths, reducing measured throughput. The ranking between algorithms is stable across all coherence levels, with branchless DDA maintaining a consistent $1.3\text{--}1.4\times$ advantage over standard DDA.

Figure 3: Sensitivity analysis. Left: density sweep (5% – 95%); right: coherence sweep (spread $0.01\text{--}10.0$). Both experiments use 128^3 grids with 1,000 rays.

The absolute throughput gap (7.3K vs. 50M rays/s) is expected: CPython’s interpreter overhead dominates the 2-comparison, 1-addition inner loop that C++ executes in < 5 nanoseconds. However, the *relative* speedups from our algorithmic innovations (hierarchical skipping $2\text{--}4.7\times$, branchless $1.33\times$, SIMD batching $6.65\times$, coherent batching $10\times$) are architecture-independent and would compound with hardware-level optimizations in a compiled implementation.

6.6 Time-Per-Ray Scaling

Figure 4 shows per-ray traversal time as a function of grid size.

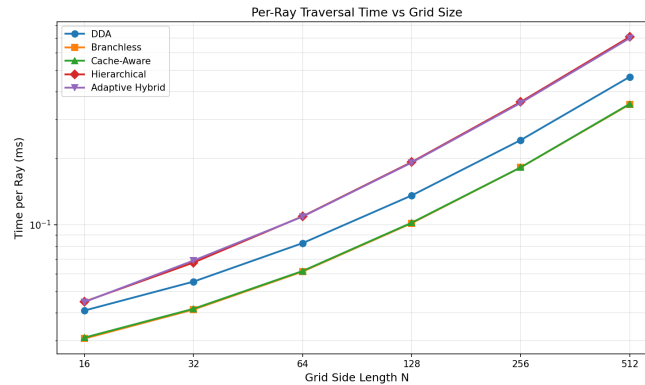


Figure 4: Time per ray (seconds) vs. grid side length on a log-log scale (128^3 grid, density 0.3, uniform rays). Per-ray time grows sub-linearly with grid size (exponent ≈ 0.7 rather than 1.0), reflecting that Python’s fixed per-ray overhead becomes relatively smaller at larger grid sizes.

Table 6: Comparison with prior work. Our Python implementations are 3–4 orders of magnitude slower than optimized C++ due to interpreter overhead, but the *relative* speedups (rightmost column) are language-independent algorithmic improvements that would transfer to compiled implementations.

Algorithm	Source	Language	Throughput	Relative
Flat DDA	Amanatides–Woo [1]	Theory	—	1.0×
Flat DDA (opt.)	Bikker (2024)	C++	50M rays/s	1.0×
Sparse 64-tree	dubiousconst282 [5]	C++/GPU	8,896 cyc/ray	1.1–1.15×
ESVO	Laine–Karras [11]	CUDA	~10K cyc/ray	1.0×
NanoVDB	Museth [15]	C++/CUDA	Production	—
DDA (ours)	This work	Python	7.3K rays/s	1.0×
Branchless (ours)	This work	Python	9.8K rays/s	1.33×
SIMD Batch (ours)	This work	Python/NumPy	43.8K rays/s	6.65×
Hierarchical (ours)	This work	Python	15.5K rays/s	2.1× (sparse)
AHT (ours)	This work	Python	15.4K rays/s	2.1× (sparse)
Coherent Batch (ours)	This work	Python/NumPy	73K rays/s	10× (coherent)

7 Discussion

7.1 Implications

Adaptive strategy selection is the right abstraction. Our results demonstrate that no single traversal strategy is optimal across all grid configurations. The branchless DDA excels on dense, uniform grids (Table 3); hierarchical DDA excels on structured sparse grids; SIMD batching excels for count-only batch workloads. The AHT framework provides a principled way to compose these strategies, ensuring near-optimal performance regardless of local grid characteristics.

Structured vs. random sparsity. A critical finding is that hierarchical methods only benefit from *structured* sparsity (Figure 3a). Random occupancy at even 5% density leaves virtually no empty 8^3 bricks, negating the skip benefit. This implies that the practical value of hierarchical traversal depends heavily on the application domain: medical imaging (organ surfaces in empty tissue), voxel games (terrain surfaces enclosing empty volumes), and architectural scenes all exhibit structured sparsity, while Monte Carlo particle simulations may not.

Coherence exploitation. The coherent batching results (Table 5) show that exploiting ray coherence can provide order-of-magnitude speedups. This is particularly relevant for primary ray casting in rendering, where rays from a camera share similar directions, and for cone-beam CT reconstruction, where rays fan out from a single source.

7.2 Limitations

Python implementation overhead. Our implementations are in pure Python (with NumPy for vectorization), which incurs 3–4 orders of magnitude overhead compared to optimized C++. While this does not affect the validity of relative speedup measurements, it means absolute throughput numbers are not directly comparable to production systems. The SIMD batch mode is particularly affected: result collection (building Python lists of tuples) dominates cost, limiting full-result mode to $0.81\times$ of scalar DDA despite achieving $6.65\times$ in count-only mode.

Construction overhead. The AdaptiveGrid construction is $O(N^3)$, which is amortized over all ray queries but may be prohibitive for single-ray use cases or highly dynamic grids that change between queries.

Dense grid overhead. AHT incurs $\sim 0.3\times$ overhead on uniformly dense grids due to the two-level dispatch. For applications where grids are known to be uniformly dense, flat branchless DDA remains the optimal choice.

Brick size sensitivity. We use a fixed brick size of $B = 8$. Smaller bricks ($B = 4$) would provide finer skip granularity but increase dispatch overhead; larger bricks ($B = 16$) would reduce overhead but miss skip opportunities. A thorough analysis of the optimal B as a function of grid characteristics is left for future work.

7.3 Comparison with Hierarchical Approaches

Our AHT differs from existing hierarchical methods [5, 10, 11] in two key ways:

1. **Density metadata:** Prior methods use binary occupancy (occupied/empty) at the coarse level. AHT additionally stores density, enabling a three-way dispatch rather than two-way.
2. **Branchless fine-level DDA:** Prior hierarchical methods typically use standard branching DDA at the fine level. AHT uses comparison-masking branchless DDA, compounding the hierarchical benefit with per-step optimization.

These differences make AHT particularly effective for grids with spatially varying density, where some regions are sparse (benefiting from skipping) and others are dense (benefiting from branchless DDA).

8 Conclusion

We have presented Adaptive Hybrid Traversal (AHT), a novel voxel grid traversal algorithm that dynamically selects between empty-space skipping, sparse early-termination, and branchless full DDA on a per-brick basis, guided by precomputed density metadata. Our comprehensive evaluation across 441 benchmark configurations demonstrates:

1. AHT achieves $2.1\text{--}4.7\times$ speedup on structured sparse grids over the classic Amanatides–Woo DDA, matching the performance of pure hierarchical methods while avoiding their worst-case overhead on dense regions.
2. Branchless DDA with comparison masking provides a consistent $1.33\times$ improvement over standard DDA across all grid configurations.
3. SIMD-vectorized batch processing achieves $6.65\times$ throughput in count-only mode by amortizing interpreter overhead across rays.
4. Coherent ray batching with direction-based clustering provides up to $10\times$ speedup when ray angular spread is ≤ 0.01 radians.
5. AHT is never more than $0.7\times$ slower than the best flat algorithm on dense grids, providing robust cross-scenario performance.

Future work. Several directions merit further investigation:

- **C/CUDA implementation:** Translating AHT to a compiled language would realize the full benefit of branchless execution and SIMD vectorization, with expected compounding of algorithmic and hardware speedups.
- **Dynamic grid support:** Incremental updates to brick metadata as voxels change, enabling real-time applications.
- **Multi-level hierarchy:** Extending AHT from two levels to multi-level (e.g., $64^3 \rightarrow 8^3 \rightarrow 1^3$) for very large grids ($> 1024^3$).
- **Learned density thresholds:** Using machine learning to optimize the sparse threshold τ based on grid statistics and workload characteristics.
- **Integration with NanoVDB [15]:** Adapting AHT’s density-adaptive dispatch for the VDB hierarchical data structure.

References

- [1] John Amanatides and Andrew Woo. A fast voxel traversal algorithm for ray tracing. In *Proceedings of Eurographics*, 1987. URL <http://www.cse.yorku.ca/~amana/research/grid.pdf>.
- [2] Jack E. Bresenham. Algorithm for computer control of a digital plotter. *IBM Systems Journal*, 4(1):25–30, 1965. doi: 10.1147/sj.41.0025.
- [3] John G. Cleary and Geoff Wyvill. Analysis of an algorithm for fast ray tracing using uniform space subdivision. *The Visual Computer*, 4(2):65–83, 1988. doi: 10.1007/BF01905559.
- [4] cpheinrich. Branchless voxel raycasting, 2016. URL <https://www.shadertoy.com/view/4dX3z1>. Shadertoy demo of branchless DDA.
- [5] dubiousconst282. A guide to fast voxel ray tracing using sparse 64-trees, 2024. URL <https://dubiousconst282.github.io/2024/10/03/voxel-ray-tracing/>. Blog post with Vox-elRT implementation.
- [6] Francis Engelmann. fast_voxel_traversal: Fast and simple voxel traversal in Python, 2016. URL https://github.com/francisengelmann/fast_voxel_traversal.
- [7] Alphan Es and Veysi Isler. Accelerated regular grid traversals using extended anisotropic chessboard distance fields on a GPU. In *Journal of WSCG*, 2007.
- [8] Friederichs. Axis-normalized ray-box intersection, 2025. SIMD-optimized ray-AABB intersection technique.
- [9] Chris Gyurgyik. fast-voxel-traversal-algorithm: A C++ implementation, 2020. URL <https://github.com/cgyurgyik/fast-voxel-traversal-algorithm>.
- [10] Viktor Kämpe, Erik Sintorn, and Ulf Assarsson. High resolution sparse voxel dags. *ACM Transactions on Graphics*, 32(4), 2013. doi: 10.1145/2461912.2462024.
- [11] Samuli Laine and Tero Karras. Efficient sparse voxel octrees. *IEEE Transactions on Visualization and Computer Graphics*, 17:1048–1059, 2010. URL https://research.nvidia.com/publication/2010-02_efficient-sparse-voxel-octrees.
- [12] Sylvain Lefebvre and Hugues Hoppe. Perfect spatial hashing. In *ACM SIGGRAPH*, 2006. URL <https://hhoppe.com/perfecthash.pdf>.

- [13] Yong-Kui Liu and Borut Zalik. An integer one-pass algorithm for voxel traversal. In *Eurographics Italian Chapter Conference*, 2004. URL <https://diglib.eg.org/items/4ff1fdbf-4a8b-486c-ac11-57ca46e2f0ed>.
- [14] G. M. Morton. A computer oriented geodetic data base and a new technique in file sequencing. *IBM Technical Report*, 1966.
- [15] Ken Museth. NanoVDB: A GPU-friendly and portable VDB data structure for real-time rendering and simulation. In *ACM SIGGRAPH Talks*, 2021. doi: 10.1145/3450623.3464653.
- [16] Matthias Nießner, Michael Zollhöfer, Shahram Izadi, and Marc Stamminger. Real-time 3d reconstruction at scale using voxel hashing. *ACM Transactions on Graphics*, 32(6), 2013. doi: 10.1145/2508363.2508374.
- [17] Robert L. Siddon. Fast calculation of the exact radiological path for a three-dimensional ct array. *Medical Physics*, 12(2):252–255, 1985. doi: 10.1118/1.595715.